

Exam 1 for COMS 4776
Neural Networks and Deep Learning
Fall 2025
Thursday, October 16, 2025

Name: Alisdair Krettinger-Walters

Uni: adl2164

This is a closed-book test. It is marked out of 100 points. Please answer ALL of the questions. Here is some advice:

- The questions are NOT arranged in order of difficulty, so you should attempt every question.
 - Questions that ask you to “explain your answer” something only require short (1-3 sentence) explanations. Don’t write a full page of text. We’re just looking for the main idea.
-
- None of the questions require long derivations. If you find yourself plugging through lots of equations, consider giving less detail or moving on to the next question.
 - Some questions may have more than one right answer.
 - You can use the back of the pages if you need more space for your answers.

Q1: 27 / 30Q2: 15 / 20Q3: 11 / 13Q4: 13 / 13Q5: 22 / 24Final mark: 88 / 100

1. Give short answers for each of the following (5 points each).

- ★ (a) What is a potential problem caused by tuning hyperparameters using the test dataset?

Hyper parameter tuning on the test set might

not reflect the hyperparameters best suited for the true distribution,

→ In order to address this, cross-fold validation randomly breaks up the training & validation set, which mitigates this risk.

→ and will likely give inflated values when final analysis & model criticism occurs

- (b) Consider a multivariate regression problem in which the units take on quite different values, e.g., we predict the height of an individual in feet and their weight in pounds. What problem do you see this causing? Propose a solution to this problem.

1) Ridge/Lasso normalization with shared lambdas would disproportionately move weights to 0, without taking unit/scale into account

2) Fixing this by mean & stdev scaling would address the issue

- (c) What is the motivation for using early stopping during neural network training? Describe a potential problem with it.

We use early stopping to freeze training once the network has started to exhibit overfitting. It can be problematic when the ³ validation loss curve is very "bumpy", causing us to conclude training artificially early

assuming
test !=
training

- Age?
Race?
- (d) Consider a convolutional neural network (CNN) trained on a large dataset of human faces for identity recognition. You now want to apply transfer learning to a new task. Describe (1) a new task in which transfer learning might work well; (2) which layers you would likely fine-tune or freeze; (3) a task in which you expect transfer learning would not work well.

1) Any task involving facial data (age prediction, race prediction, familial clustering) would probably transfer well

2) Fine tuning the final layers & freezing the lower layers would likely work well

3) Cat vs Dog recognition

- (e) For the purpose of downsampling, explain the fundamental difference between using a 2×2 max pooling layer and using another convolutional layer with a stride of 2. What is a key advantage of each method?

The max pooling layer reduces a 2×2 grid $\rightarrow 1 \times 1$ by selecting (for example) the max feature. This typically would occur post convolution, to approximate a larger region by a single value.

A convolutional layer w/ stride = 2 would shift the kernel two spots between measurements, also reducing the dimension of the output layer.

Pooling's advantage is selecting the most important feature from a larger space to achieve reduction, while striding achieves this by downsampling that first measurement. The cost/benefit of each would probably be decided by OOS relative performance in training & validation

- (f) You train two neural networks with L2 regularization with different regularization parameters λ_1 and λ_2 and plot their validation loss during training. The network trained with λ_1 shows some decline in validation loss at the beginning of training, but then the loss sharply increases for the rest of training. The network trained with λ_2 shows a slight decrease in validation loss that then plateaus at a high value. What kinds of settings (i.e. high or low) for λ_1, λ_2 could result in this behavior? Justify your answer for each case.

λ_1 is likely lower. It exhibits quick improvements in early epochs, and quickly starts to overfit the data

λ_2 is likely very high. Slow improvements coupled w/ a high plateau indicate a very strong ridge prior that limits the network's ability to learn the data well

2. Consider training a convolutional neural network on RGB images. Let the input image size be 100×100 pixels with three color channels. There are 16 convolution filters in the first conv layer, where each kernel is 5×5 . Assume the output size of the first convolution layer is the same as the image. The stride is 1. There is no bias unit in this conv net. (20 points: 4 points each).

$100 \times 100 \times 3$

- (a) How many learnable weights are there in the first conv layer?

$$16 \times 5 \times 5 = 400$$

-2

- (b) How many scalar multiply operations do we need to compute the output of the conv layer?

weights $\times$ # kernel mults

-3

$$400 \times 300 = 120,000$$



- (c) If we increase the input image size from 100x100 to 200x200, how many more parameters do we need to add to the first conv layer?

None

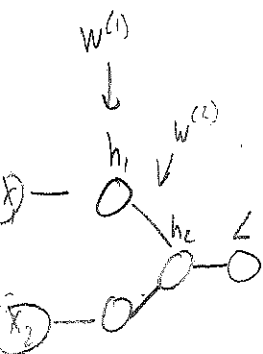
- (d) If we increase the input image from 100x100 to 200x200, how does the number of scalar multiply operations change?

It will 4x $\rightarrow H' = 2H$

$$W' = 2W$$

$$H'W' = 4(HW)$$

3. A ReLU unit is considered "dead" if it does not receive an input greater than 0 for *any* training example. Although dead ReLUs can show up in various different architectures, consider a two-hidden-layer MLP for this question.



- (a) Write an expression for the gradient of the loss with respect to the weights from the input to first hidden layer, used in the backprop algorithm. For a dead ReLU unit, what is the value of this gradient for its incoming weights and its outgoing weights? Provide a brief justification for your answer. (5 points).

$$\bar{L} = 1$$

$$\bar{h}_2 = \bar{L} \cdot \frac{dL}{dh_2}$$

$$\bar{h}_1 = \bar{h}_2 \cdot \frac{dh_2}{dh_1}$$

$$h = \text{ReLU}(w^{(1)}x + b)$$

$$\begin{aligned} \bar{w}^{(1)} &= \bar{h}_1 \cdot \frac{dw}{dw} \\ &= \bar{h}_1 \cdot \text{ReLU}'(x) \end{aligned}$$

The value of gradient for incoming & outgoing weights would both be 0

~~It would be 0~~

4

- (b) If a ReLU unit in the second hidden layer ever becomes dead, does it ever have the chance to get revived during training? Provide a brief justification for your answer. (5 points).

★

~~It cannot, unless the model is fed new unseen data that can revive the unit. This is because the p.d. of the~~

~~Yes, it can. The layer before can start outputting~~

No, it can't. The layer before cannot learn in backprop now, and its output values will never change enough to revive the dead unit.

4

- (c) You notice that in your particular problem, ReLU units initialized with large, positive weights tend to stay active over the course of training. To promote this, you initialize all of the incoming and outgoing weights of your ReLU units with the same large positive constant c . What issue could this initialization scheme produce? (3pts)

This would severely handicap your network.

The grad of each ReLU per row would be the same, and would update in sync. You've effectively reduced the width of your net & compromised expressiveness

3

4. Recall the softmax cross entropy loss of D classes is given by:

$$\mathcal{L} = - \sum_{i=1}^D t_i \log y_i, \quad \text{where } y_i = \frac{\exp\{z_i\}}{\sum_{j=1}^D \exp\{z_j\}}$$

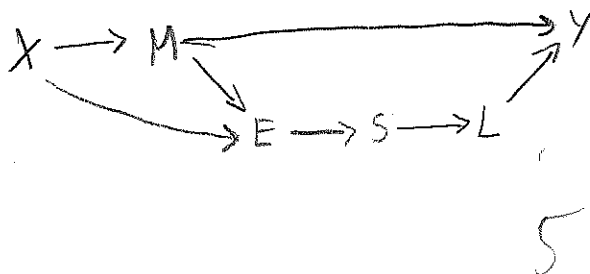
The above expression can be simplified in terms of the logits:

$$\mathcal{L} = - \sum_{i=1}^D t_i \log \frac{\exp\{z_i\}}{\sum_{j=1}^D \exp\{z_j\}} = - \sum_{i=1}^D t_i z_i + \log \sum_{j=1}^D \exp\{z_j\}$$

In practice, the logits, z , can take on very small values, which causes numerical instability in the second term, $\log \sum_{j=1}^D \exp\{z_j\}$. (Taking the log of a value close to zero needs to be avoided.) Most of the machine learning libraries use a numerically stable implementation for the second term, called “LogSumExp”:

```
def logsumexp(X):
    M = np.max(X)
    E = np.exp(X - M)
    S = np.sum(E)
    L = np.log(S)
    Y = M + L
    return Y
```

- (a) Draw the computation graph of LogSumExp in terms of the intermediate variables, X, M, E, S, L, Y . (5 points).



- (b) Write down the backprop updates for the whole LogSumExp function, i.e., $\bar{X}$ given $\bar{Y}$. Notice Y is a scalar (You may give your answers assuming either $D=2$ or for the more general case.) (8 points).

$$\bar{Y} = 1$$

$$\bar{L} = \bar{Y} \cdot \frac{dy}{dL}$$

$$= \bar{Y} \cdot 1 \quad \checkmark$$

$$\bar{S} = \bar{L} \cdot \frac{dL}{dS} \quad \checkmark$$

$$= \bar{L} \cdot \frac{1}{S}$$

$$S = E_1 + E_2 + E_3$$

$$\bar{E} = \bar{S} \cdot \frac{dS}{dE} \quad \checkmark$$

$$= \bar{S} \cdot 1$$

$$\bar{M} = \bar{Y} \cdot \frac{dy}{dM} + \bar{E} \cdot \frac{dE}{dM} \quad \checkmark$$

$$E = e^{x-M} = \frac{e^x}{e^M} = e^x e^{-M} = e^x e^{-M}$$

$$= \bar{Y} \cdot 1 + \bar{E} \cdot -M e^x e^{-M}$$

$$\bar{X} = \bar{E} \cdot \frac{dE}{dx} + \bar{M} \cdot \frac{dM}{dx} \quad \checkmark$$

$$= \bar{E} \cdot x e^x e^{-M} + \bar{M} \cdot \max'(x)$$

8

trickier derivative representation

5. You would like to construct a neural network classifier that takes as input a photograph from a safari, and outputs a label of which animal is the main subject in the image, where the classes are: rhino, hippo, giraffe, hyena, elephant. (24 points: 4 points each).

- (a) What activation function for the network output layer is the most appropriate one for this problem, and why?

Softmax is the best,

1) Gives 1 unique solution $\rightarrow [0, 0, 1, 0, 0]^T$

2) is differentiable

4

- (b) You decide to use cross entropy loss to train your network. Recall that the cross-entropy loss for a single example is defined as follows:

$$\mathcal{L}(\mathbf{y}, \mathbf{t}) = - \sum_{i=1}^D t_i \log y_i$$

where $\mathbf{y}$ is the predicted probability distribution over the classes and $\mathbf{t}$ represents the ground truth vector, which is zero everywhere except for the correct class.

After training the model, you find it achieves 100% accuracy on the training data, where the accuracy is 1 if the maximum model prediction matches the ground-truth class, and 0 otherwise. However, you observe that the training loss does not quite reach zero. Can the training loss reach zero for your chosen activation function? Describe why or why not.

It _↓ can't reach 0. CE loss penalizes proportionately to the difference between $\mathbf{y}$ & $\mathbf{t}$, so as long as the predicted score for any single training example (non-target) is non-zero, there

4

will be a loss. So, if the net gives any weight to rhino when the answer is hippo, etc, there will be non-zero training loss

- (c) Now you decide to change the task for the model. Each image can contain multiple types of animals (e.g., a rhino and a giraffe), and your goal is to label each image as to whether or not there is an instance of each of the 5 classes in it. How would this change the ground-truth image labels for the task?

You would now have $\sum_{i=1}^D t_i \neq 1$. The image labels

could contain up to D "correct" answers.

$$t \in \mathbb{R}^{D \times 1}$$

#2

$$0 \leq \sum_{i=1}^D t_i \leq D$$

need to mention target label of 1 for correct animal

- (d) To avoid extra work, you decide to retrain a new model with the same activation and loss function. Explain why this is problematic.

Softmax & CE loss are designed with the constraint

$$\sum_{i=1}^D t_i = 1$$

In mind. Violating that assumption requires a completely different activation & loss to see good results

4

- (e) Propose a combination of activation function for the last layer and loss function, which is better suited for this multi-class labeling task.

You could output a D dimensional vector, for which each value is computed by a logistic activation function & CE (binary simplified version still works) loss.

4

- (f) How could you modify the activation function in the output layer in (a) to handle the case where none of the 5 classes are present in a given image?

I propose adding a $D+1$ class label "none" that is the correct answer when no objects exist

4